

Proyecto Integrador: Diseño de Arquitectura Híbrida para Ecommify

Asignatura: Diseño y Optimización de Bases de Datos

Actividad: Guía de Actividad Evaluativa Unidad 2

Fecha: Semana del 4 al 10 de Mayo

Integrantes del Equipo:
Juan Daniel Valderrama Pérez
Jorge Esteban Triviño Correa
Javier Andres Baron Fontanilla

Docente: Miguel Alfonso Varela Fonseca

Universidad de la Sabana
Facultad de Ingeniería

Tabla de Contenido

Enfoque Arquitectónico Seleccionado	4
Abstract	4
Análisis de Requisitos	4
• Contexto de Ecomimfy	4
• Requisitos funcionales	5
• Requisitos no funcionales	5
• Identificación de entidades del dominio	6
◦ PostgreSQL - Transaccional	6
• MongoDB - Documental	6
• Tabla de volúmenes de datos esperados	6
• Justificación arquitectónica	7
Diseño Conceptual	8
• Diagrama ER completo de alta calidad - Sin Normalizar	8
Figura 1. Diagrama ER Sin Normalizar del Dataset Principal Ollist.	8
• Descripción detallada de cada entidad y relación	8
• Descripción de Relaciones	9
• Restricciones de negocio identificadas	9
◦ Integridad Transaccional - Pedidos y Pagos	9
◦ Logística y Vendedores	9
◦ Calidad y Experiencia - Reviews	9
◦ Catálogo de Productos	9
Diseño lógico - Módulo PostgreSQL	10
• Esquema relacional normalizado	10
Figura 2. Diagrama ER Normalizado (3FN) para el Diseño Propuesto del Módulo PostgreSQL.	12
• Aplicación de tipos avanzados	13
• Justificación técnica de tipos avanzados	13
• JSONB	13
• Arrays	13
• TSTZRANGE	13
• Extensiones a utilizar con justificación.	14
• Justificación de extensiones	14
◦ pg_trgm	14
• PostGIS	14
• Estrategia de particionamiento	14
• Particiones mensuales:	14
• Justificación del particionamiento	15
Diseño lógico preliminar - Módulo MongoDB	15
• Colecciones principales identificadas	15
• Esquema de documentos - Estructura Simplificada	15
• 1. Colección <code>products_catalog</code>	15
• 2. Colección <code>reviews</code>	16
• 3. Colección <code>user_sessions</code>	17
• 4. Colección <code>analytics_events</code>	18
• Patrones de modelado a aplicar	18

1. Attribute Pattern - Patrón de Atributos	18
2. Subset Pattern - Patrón de Subconjunto	18
3. Extended Reference Pattern - Referencia Extendida	18
4. Patrón de Expiración - TTL - Time To Live	18
5. Time-Series Collections	18
Decisiones arquitectónicas justificadas	19
• Matriz de decisión por entidad	19
• Análisis en profundidad del Teorema CAP y PACELC	19
Figura 3. Análisis de Aplicación del Teorema CAP en el Diseño Híbrido Propuesto para el Dataset Ollist.	20
1. PostgreSQL: Enfoque CP y PC en(PACELC)	20
2. MongoDB: Enfoque AP y PA/EL en (PACELC)	20
• Evaluación de Trade-offs y Estrategias de Mitigación	21
• Flujos de sincronización (PostgreSQL-MongoDB)	21
Anexos	22
• Diccionario de datos	22
• Scripts SQL preliminares	22
○ Creación de extensiones	22
• Índices	22
• Trigger updated_at	23
• Ejemplos de datos	23
○ customers	23
• orders	23

Enfoque Arquitectónico Seleccionado

El equipo decide seleccionar la Arquitectura Políglota Híbrida, Opción 3 planteada en la guía previa. La solución propuesta es entonces híbrida basada en el Teorema CAP, buscando un balance entre consistencia y disponibilidad. Justificación de la Arquitectura:

- PostgreSQL: Principal almacenamiento para transacciones donde la integridad está en primer plano, como pagos y estados de envío.
- MongoDB: Elegido Almacenamiento de documentos para el catálogo, permitiendo que diferentes categorías de productos tengan atributos variables sin modificar el esquema global.

Abstract

Este documento presenta el diseño de la arquitectura híbrida de bases de datos para Ecommify, orientada a soportar cargas transaccionales (OLTP) y analíticas (OLAP). Mediante un enfoque de persistencia políglota, se distribuye el dominio de datos entre PostgreSQL y MongoDB para equilibrar consistencia, escalabilidad y disponibilidad. Las decisiones arquitectónicas clave incluyen:

- Núcleo Transaccional - PostgreSQL: Las entidades de Orders, Payments, Inventory y Customers se asignaron a este motor relacional. Esta decisión garantiza el cumplimiento estricto de propiedades ACID, integridad referencial y prevención de condiciones de carrera mediante bloqueos a nivel de fila durante operaciones críticas.
- Catálogo y Analítica - MongoDB: El Products Catalog, Reviews y Sessions se delegaron a este motor NoSQL. Su estructura polimórfica resuelve la heterogeneidad de los productos y optimiza la latencia en escenarios con un ratio de lectura/escritura superior a 100:1. Adicionalmente, facilita la retención de datos analíticos mediante índices TTL y soporte para series de tiempo.

La segregación estratégica garantiza que el sistema escale horizontalmente en la capa de consultas, manteniendo al mismo tiempo una consistencia inquebrantable en las operaciones financieras del negocio.

Análisis de Requisitos

- **Contexto de Ecomimfy**

Ecommify es una plataforma de comercio electrónico híbrida orientada a soportar operaciones transaccionales de alta concurrencia y procesamiento analítico de datos derivados de comportamiento de usuarios, catálogo de productos y trazabilidad logística.

La plataforma debe soportar:

- Gestión de clientes.
- Administración de pedidos.
- Procesamiento de pagos.
- Gestión de inventario.
- Catálogo dinámico de productos.
- Sistema de reseñas.
- Consultas analíticas y dashboards operacionales.

Debido al comportamiento heterogéneo de los datos y los diferentes patrones de acceso, se propone una arquitectura políglota que combina PostgreSQL y MongoDB, aprovechando las fortalezas de ambos motores.

- **Requisitos funcionales**

ID	Requisito
RF-01	Registrar clientes y autenticarlos en la plataforma
RF-02	Gestionar catálogo de productos con atributos dinámicos
RF-03	Permitir creación y actualización de pedidos
RF-04	Procesar pagos asociados a pedidos
RF-05	Gestionar inventario por vendedor
RF-06	Permitir reseñas y calificaciones de productos
RF-07	Consultar historial de compras
RF-08	Generar reportes y métricas comerciales
RF-09	Permitir búsquedas tolerantes a errores tipográficos
RF-10	Gestionar promociones y periodos de descuento

- **Requisitos no funcionales**

Categoría	Requisito
Escalabilidad	Soportar crecimiento horizontal del catálogo y consultas concurrentes
Consistencia	Garantizar integridad ACID en pedidos y pagos
Disponibilidad	Mantener acceso continuo al catálogo y reseñas
Rendimiento	Optimizar consultas OLTP y OLAP
Flexibilidad	Soportar atributos dinámicos de productos
Seguridad	Garantizar integridad referencial y control de acceso
Observabilidad	Permitir monitoreo de transacciones y consultas
Mantenibilidad	Facilitar evolución del esquema y nuevas funcionalidades

- **Identificación de entidades del dominio**

- PostgreSQL - Transaccional

Entidad	Descripción
customers	Información de clientes
orders	Pedidos realizados
order_items	Productos asociados a pedidos
payments	Información de pagos
sellers	Vendedores
products	Inventario de productos disponible
geolocation	Información geográfica

- MongoDB - Documental

Colección	Descripción
products_catalog	Catálogo flexible de productos
reviews	Reseñas y comentarios
user_sessions	Sesiones y comportamiento
analytics_events	Eventos de navegación

- **Tabla de volúmenes de datos esperados**

Entidad	Volumen estimado	Crecimiento esperado	Tipo de carga
orders	10M+ registros	Alto	OLTP
order_items	50M+ registros	Alto	OLTP
payments	10M+ registros	Alto	OLTP
customers	2M+ registros	Medio	OLTP
products_catalog	500K+ documentos	Alto	Lectura intensiva
reviews	20M+ documentos	Alto	Escritura/lectura
analytics_events	500M+ eventos	Muy alto	OLAP/Eventos

- **Justificación arquitectónica**

La segmentación tecnológica responde a diferentes patrones de acceso y atributos de calidad.

PostgreSQL se utiliza en módulos de transacción y con una relación clara debido a:

- Soporte ACID,
- Integridad referencial,
- Manejo robusto de transacciones,
- Particionamiento,
- Extensiones avanzadas,
- Consultas complejas mediante SQL.

MongoDB se adopta para componentes con:

- Esquemas dinámicos,
- Alta variabilidad estructural,
- Consultas rápidas de lectura,
- Necesidad de escalabilidad horizontal.

Esta aproximación sigue un modelo de persistencia políglota alineado con la arquitectura propuesta para el e-commerce.

Diseño Conceptual

● Diagrama ER completo de alta calidad - Sin Normalizar

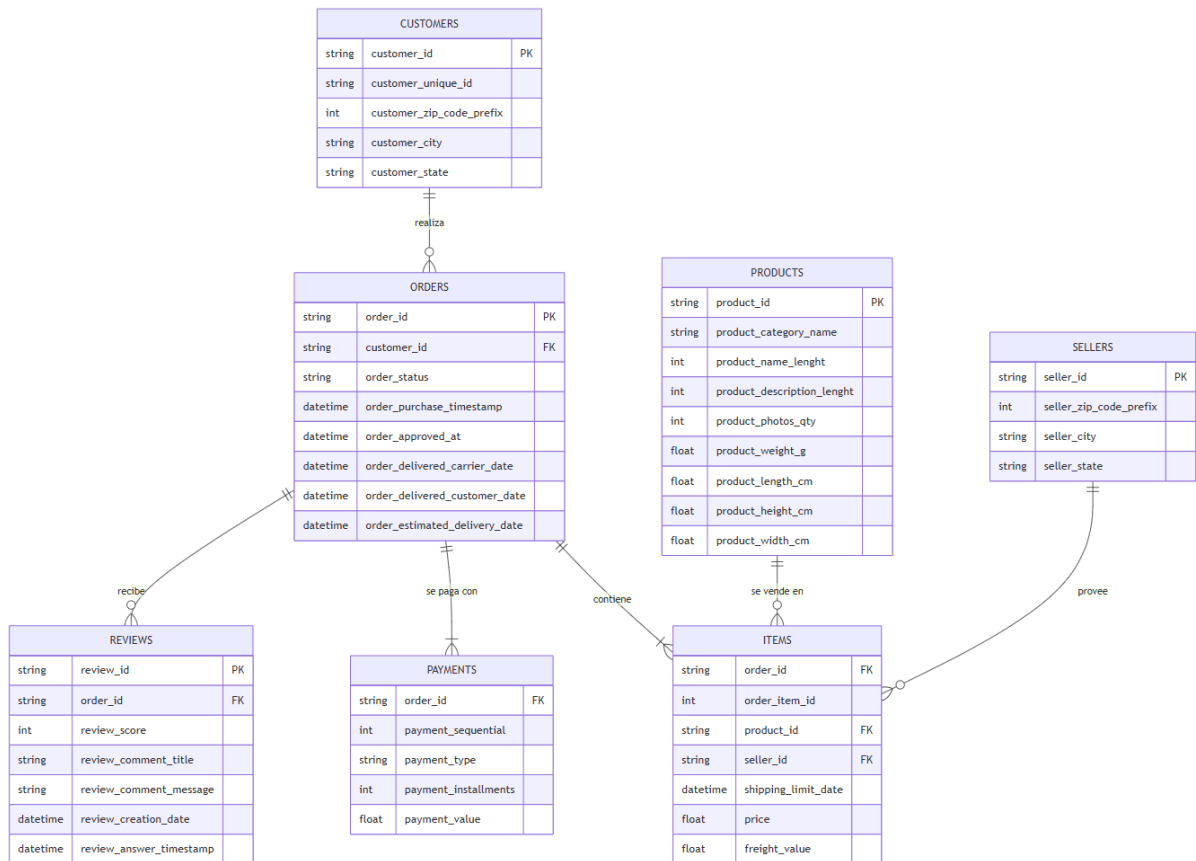


Figura 1. Diagrama ER Sin Normalizar del Dataset Principal Ollist.

● Descripción detallada de cada entidad y relación

- **Customers:** Representa a los compradores. Contiene el `customer_id`, para la transacción y el `customer_unique_id`, para seguimiento histórico del usuario.
- **Orders:** Registra el estado del ciclo de vida de la compra, creado, entregado, etc. Los tiempos de procesamiento.
- **Items:** Entidad asociativa, es el detalle del pedido. Desglosa cada producto dentro de una orden, incluyendo su precio y costo de envío.
- **Products:** Catálogo de productos. Almacena características físicas y categorías, esenciales para la logística y el análisis de catálogo.
- **Sellers:** Datos de los vendedores que ofrecen sus productos en la plataforma, permitiendo la gestión de un modelo multivendedor.
- **Payments:** Registra la dimensión financiera. Permite múltiples transacciones o métodos de pago para una sola orden.
- **Reviews:** Almacena la experiencia del usuario. Vincula la satisfacción del cliente directamente con un pedido específico.

- **Descripción de Relaciones**

- **Customers (1) — (N) Orders:** Un cliente puede realizar múltiples pedidos a lo largo del tiempo, pero cada pedido pertenece a un único cliente.
- **Orders (1) — (N) Items:** Un pedido puede contener uno o varios productos. La relación es obligatoria para que el pedido exista.
- **Orders (1) — (N) Payments:** Un pedido puede pagarse mediante uno o más métodos (ej. crédito y cupón).
- **Orders (1) — (N) Reviews:** Un pedido puede recibir una o más reseñas, dependiendo de si se evalúa el servicio o productos por separado.
- **Products (1) — (N) Items:** Un producto puede ser vendido muchas veces en diferentes pedidos.
- **Sellers (1) — (N) Items:** Un vendedor puede suministrar múltiples productos en distintos pedidos dentro de la plataforma.

- **Restricciones de negocio identificadas**

- Integridad Transaccional - Pedidos y Pagos
 - Sincronía de Pagos: El valor total registrado en `order_payments` debe coincidir exactamente con la suma de `price + freight_value` en `order_items`. No puede haber una orden pagada por un monto distinto al de sus productos.
 - Métodos de Pago: Solo se permiten tipos de pago predefinidos, `boleto`, `credit_card`, `voucher`, `debit_card`. Cualquier otro debe ser rechazado por el sistema.
 - Secuencia de Tiempo: La fecha de entrega `order_delivered_customer_date` no puede ser anterior a la fecha de compra `order_purchase_timestamp`.
- Logística y Vendedores
 - Geolocalización Obligatoria: No se debe permitir el registro de un customer o seller cuyo `zip_code_prefix` no exista previamente en la tabla de geolocation. Esto asegura que siempre se pueda calcular el flete.
 - Límite de Envío: La fecha `shipping_limit_date` en los ítems es una restricción a considerar, si el vendedor no entrega al transportista antes de esa fecha, el pedido debe marcarse con una alerta de retraso.
- Calidad y Experiencia - Reviews
 - Reseña Post-Venta: Una review solo puede ser vinculada a una orden que ya tenga un estado de `delivered-entregado` o `shipped-enviado`, evitando reseñas falsas de productos no recibidos.
 - Rango de Calificación: El `review_score` debe estar estrictamente en el rango de 1 a 5.
- Catálogo de Productos
 - Dimensiones Positivas: El peso `product_weight_g` y las dimensiones `length`, `height`, `width`, deben ser valores mayores a cero. Un producto con peso 0 rompería el cálculo logístico.
 - Categorización: Todo producto debe tener asignada una categoría válida para poder ser indexado en el motor de búsqueda de la plataforma.

Diseño lógico - Módulo PostgreSQL

- **Esquema relacional normalizado**

El modelo relacional es diseñado considerando el dataset original y siguiendo principios de normalización hasta Tercera Forma Normal (3FN), eliminando redundancias y garantizando integridad referencial.

Las entidades transaccionales principales son:

- customers
- orders
- order_items
- payments
- sellers
- products
- geolocation

Tabla customers

```
CREATE TABLE customers (  
  customer_id UUID PRIMARY KEY,  
  customer_unique_id UUID NOT NULL,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  phone VARCHAR(20),  
  geolocation_id UUID,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
  CONSTRAINT unique_customer_email UNIQUE(customer_unique_id, email)  
);
```

Tabla products

```
CREATE TABLE products (  
  product_id UUID PRIMARY KEY,  
  sku VARCHAR(50) UNIQUE NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla geolocation

```
CREATE TABLE geolocation (  
  geolocation_id UUID PRIMARY KEY,  
  zip_code VARCHAR(15),  
  city VARCHAR(100),  
  state CHAR(2),  
  coordinates POINT  
);
```

Tabla sellers

```
CREATE TABLE sellers (  
  seller_id UUID PRIMARY KEY,  
  seller_name VARCHAR(150) NOT NULL,  
  geolocation_id UUID REFERENCES geolocation(geolocation_id),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla orders

```
CREATE TABLE orders (  
  order_id UUID NOT NULL,  
  customer_id UUID REFERENCES customers(customer_id),  
  order_status VARCHAR(30),  
  purchase_timestamp TIMESTAMP NOT NULL,  
  delivered_timestamp TIMESTAMP,  
  total_amount NUMERIC(12,2) CHECK(total_amount >= 0),  
  PRIMARY KEY (order_id, purchase_timestamp)  
) PARTITION BY RANGE (purchase_timestamp);
```

Tabla order_items

```
CREATE TABLE order_items (  
  order_item_id UUID PRIMARY KEY,  
  order_id UUID REFERENCES orders(order_id),  
  purchase_timestamp TIMESTAMP NOT NULL,  
  product_id UUID NOT NULL,  
  seller_id UUID REFERENCES sellers(seller_id),  
  quantity INTEGER CHECK(quantity > 0),  
  price NUMERIC(10,2) CHECK(price >= 0),  
  freight_value NUMERIC(10,2)  
  FOREIGN KEY (order_id, purchase_timestamp) REFERENCES  
  orders(order_id, purchase_timestamp)  
);
```

Tabla payments

```
CREATE TABLE payments (  
  payment_id UUID PRIMARY KEY,  
  order_id UUID REFERENCES orders(order_id),  
  purchase_timestamp TIMESTAMP NOT NULL,  
  payment_type VARCHAR(30),  
  payment_installments INTEGER,  
  payment_value NUMERIC(12,2),  
  FOREIGN KEY (order_id, purchase_timestamp) REFERENCES  
  orders(order_id, purchase_timestamp)  
);
```

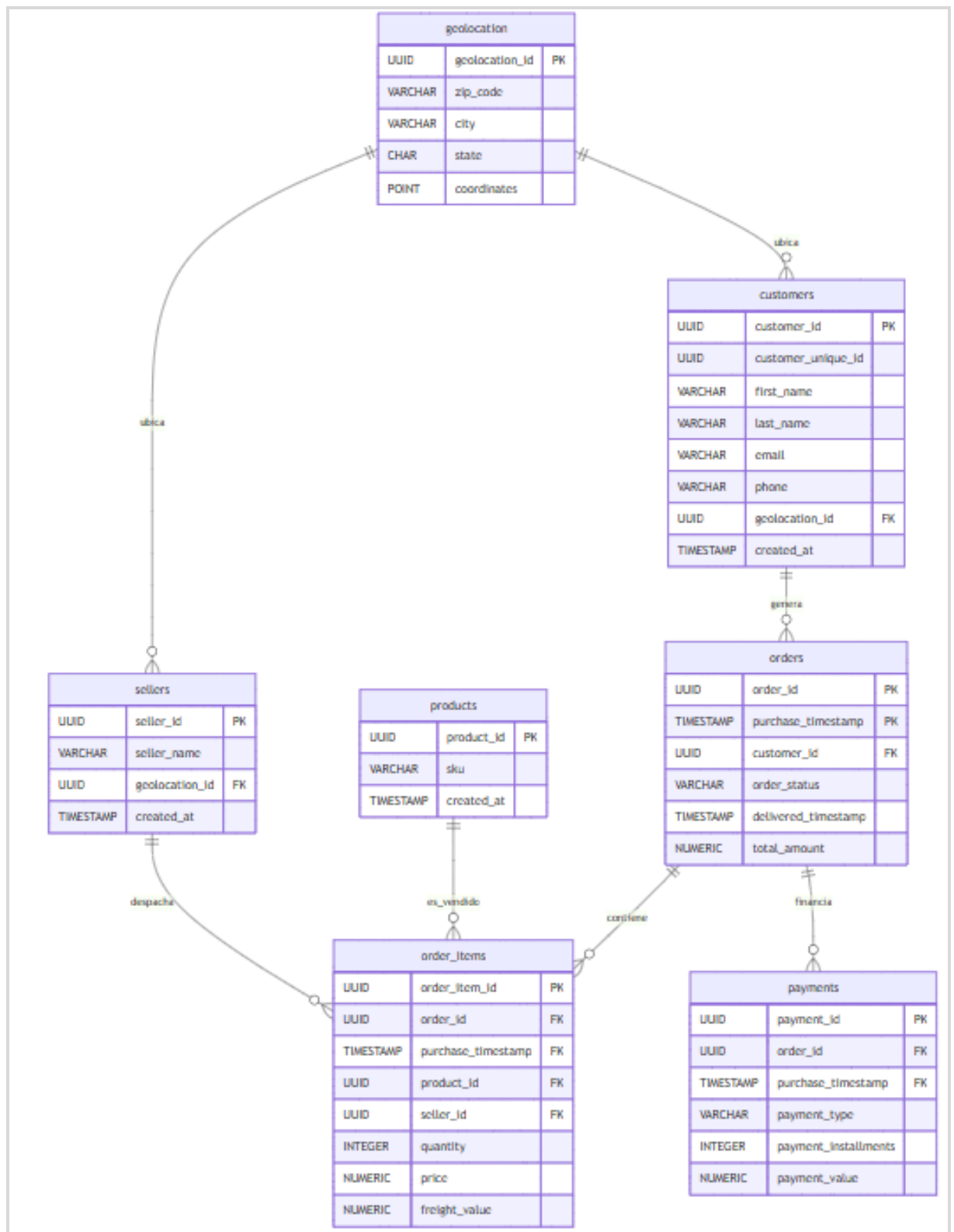


Figura 2. Diagrama ER Normalizado (3FN) para el Diseño Propuesto del Módulo PostgreSQL.

- **Aplicación de tipos avanzados**

Campo	Tipo avanzado	Justificación
product_specifications	JSONB	Atributos variables de productos
product_images	TEXT[]	Manejo eficiente de múltiples URLs
promotion_period	TSTZRANGE	Gestión temporal de promociones
metadata	JSONB	Información flexible de auditoría
coordinates	POINT/PostGIS	Consultas geoespaciales

- **Justificación técnica de tipos avanzados**

- **JSONB**

Se utiliza para almacenar especificaciones dinámicas de productos debido a la heterogeneidad del catálogo. Permite indexación GIN y consultas flexibles sin sacrificar completamente capacidades relacionales.

Ejemplo:

```
product_specifications JSONB
```

Ejemplo de contenido:

```
{
  "ram": "16GB",
  "cpu": "Ryzen 7",
  "gpu": "RTX 4060"
}
```

- **Arrays**

Se emplean para almacenar múltiples imágenes asociadas a productos:

```
product_images TEXT[]
```

Esto evita tablas auxiliares excesivas para relaciones simples 1:N de baja complejidad transaccional.

- **TSTZRANGE**

Permite representar periodos promocionales con operadores temporales eficientes:

```
promotion_period TSTZRANGE
```

Facilita consultas como:

- promociones activas

- traslapes
- vigencias temporales

- **Extensiones a utilizar con justificación.**

Extensión	Uso
pg_trgm	Búsqueda tolerante a errores
PostGIS	Consultas geoespaciales
pgcrypto	Generación segura de UUID
hstore	Metadatos clave-valor simples

- **Justificación de extensiones**

- **pg_trgm**

Permite búsquedas aproximadas en nombres de productos:

```
SELECT *  
FROM products  
WHERE product_name % 'iphne';
```

Mejora experiencia de usuario en búsquedas e-commerce.

- **PostGIS**

Permite:

- Cálculo de distancias,
- Optimización logística,
- Análisis geográfico de ventas.

Ejemplo:

```
ST_Distance()
```

- **Estrategia de particionamiento**

La tabla orders será particionada por rango temporal:

PARTITION BY RANGE (purchase_timestamp)

- **Particiones mensuales:**

- orders_2026_01
- orders_2026_02
- orders_2026_03

- **Justificación del particionamiento**

Debido al alto volumen transaccional identificado en el dataset Olist, el particionamiento:

- Reduce tiempos de consulta,
- Mejora mantenimiento,
- Optimiza VACUUM,
- Facilita archivado histórico,
- Disminuye el impacto de índices masivos.
- Además, soporta estrategias híbridas hot/cold data.

Diseño lógico preliminar - Módulo MongoDB

En el marco de nuestra arquitectura híbrida, MongoDB actuará como el motor de base de datos principal para las operaciones de lectura intensiva Heavy-Read, y el manejo de datos con esquemas variables.

- **Colecciones principales identificadas**

Como se menciona en el diseño conceptual, el ecosistema NoSQL Documental gestionará las siguientes colecciones:

Colección	Descripción
<code>products_catalog</code>	Catálogo flexible de productos
<code>reviews</code>	Reseñas y comentarios
<code>user_sessions</code>	Sesiones y comportamiento
<code>analytics_events</code>	Eventos de navegación

- **Esquema de documentos - Estructura Simplificada**

A continuación, se presentan los esquemas para todas las colecciones de MongoDB, mostrando únicamente la estructura clave, los tipos de datos y los campos requeridos.

- 1. Colección `products_catalog`:

Este documento consolida la información del producto para evitar múltiples JOINS al momento de renderizar la página de detalles.

```
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "product_id_pg": { "type": "string" },
    "category_name": { "type": "string" },
    "name": { "type": "string" },
    "description": { "type": "string" },
    "images": {
      "type": "array",
      "items": { "type": "string" }
    },
    "attributes": {
      "type": "array",

```

```

    "items": {
      "type": "object",
      "properties": {
        "k": { "type": "string" },
        "v": { "type": "string" }
      }
    }
  },
  "display_price": { "type": "number" },
  "seller": {
    "type": "object",
    "properties": {
      "seller_id_pg": { "type": "string" },
      "name": { "type": "string" },
      "reputation_score": { "type": "number" }
    }
  },
  "top_reviews": {
    "type": "array",
    "items": {
      "type": "object",
      "properties": {
        "score": { "type": "number" },
        "comment": { "type": "string" },
        "user_name": { "type": "string" }
      }
    }
  },
  "created_at": { "type": "string" },
  "updated_at": { "type": "string" }
},
"required": [
  "product_id_pg",
  "name",
  "display_price",
  "seller"
]
}

```

- 2. Colección reviews:

Documento que almacena el histórico completo de reseñas de forma desnormalizada.

```

{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "review_id_pg": { "type": "string" },
    "order_id_pg": { "type": "string" },
    "product_id_pg": { "type": "string" },
    "customer": {
      "type": "object",

```

```
{
  "properties": {
    "customer_id_pg": { "type": "string" },
    "first_name": { "type": "string" }
  },
  "score": { "type": "integer" },
  "title": { "type": "string" },
  "message": { "type": "string" },
  "creation_date": { "type": "string" }
},
"required": [
  "order_id_pg",
  "product_id_pg",
  "customer",
  "score",
  "creation_date"
]
}
```

- 3. Colección user_sessions:

Documento diseñado para rastrear sesiones activas y carritos de compra abandonados.

```
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "customer_id_pg": { "type": "string" },
    "ip_address": { "type": "string" },
    "device_info": { "type": "string" },
    "cart_items": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "product_id_pg": { "type": "string" },
          "quantity": { "type": "integer" },
          "price_at_addition": { "type": "number" }
        }
      }
    },
    "created_at": { "type": "string" },
    "last_activity": { "type": "string" }
  },
  "required": [
    "ip_address",
    "created_at",
    "last_activity"
  ]
}
```

- 4. Colección `analytics_events`:

Estructura orientada a series de tiempo Time-Series para rastrear flujos de clics y vistas.

```
{
  "type": "object",
  "properties": {
    "_id": { "type": "string" },
    "session_id": { "type": "string" },
    "event_type": { "type": "string" },
    "product_id_pg": { "type": "string" },
    "page_url": { "type": "string" },
    "event_metadata": { "type": "object" },
    "timestamp": { "type": "string" }
  },
  "required": [
    "session_id",
    "event_type",
    "timestamp"
  ]
}
```

- **Patrones de modelado a aplicar**

1. Attribute Pattern - Patrón de Atributos:

Utilizado en la colección `products_catalog` en el campo `attributes`. En lugar de tener un esquema rígido por cada tipo de producto, estandarizamos un array de clave-valor (k y v). Esto permite indexar el campo `attributes.k` y `attributes.v` para realizar filtros dinámicos de forma altamente eficiente.

2. Subset Pattern - Patrón de Subconjunto:

Implementado al incrustar el arreglo `top_reviews` dentro del documento del producto. Al cargar la página del producto, el usuario verá inmediatamente las reseñas destacadas sin necesidad de hacer una consulta adicional a la colección `reviews`.

3. Extended Reference Pattern - Referencia Extendida:

En lugar de guardar solo el `seller_id_pg`, guardamos también el nombre y la reputación del vendedor dentro del documento del producto para evitar múltiples saltos de consulta.

4. Patrón de Expiración - TTL - Time To Live:

Se aplica un índice TTL en el campo `last_activity` de la colección `user_sessions`. Si un usuario abandona su carrito y no hay actividad en 7 días, MongoDB eliminará el documento automáticamente, liberando espacio de forma autónoma.

5. Time-Series Collections:

Para la colección `analytics_events`, se utilizará la funcionalidad nativa de Time-Series de MongoDB, la cual optimiza el almacenamiento en disco y mejora el rendimiento de consultas agregadas sobre rangos de tiempo, ideal para paneles de Business Intelligence por ejemplo.

Decisiones arquitectónicas justificadas

- **Matriz de decisión por entidad**

Entidad / Dominio	Motor Seleccionado	Carga / Patrón	Justificación Técnica
Orders	PostgreSQL	OLTP (Write-Heavy)	Registro del ciclo de vida de la compra. Requiere aislamiento y particionamiento por rango <code>purchase_timestamp</code> para optimizar el rendimiento histórico sin degradar transacciones activas.
Payments	PostgreSQL	OLTP (Write-Heavy)	Dimensión financiera que soporta múltiples transacciones y cuotas por orden. Exige atomicidad absoluta para mitigar riesgos de fraude o inconsistencias en pasarelas de pago.
Customers / Sellers	PostgreSQL	OLTP (Read/Write Mixto)	Gestión de actores del ecosistema multivendedor. Requiere normalización (3FN) conectada a geolocation para auditoría logística, e integridad compuesta para tracking histórico.
Products	PostgreSQL	OLTP (Read-Heavy)	Tabla maestra simple cuyo único propósito es servir de ancla relacional e impedir que se procesen pedidos de ítems inexistentes.
Products Catalog	MongoDB	OLAP/Lecturas	Estructura polimórfica. Beneficia la latencia baja en consultas por índice de atributos y búsqueda de texto. Ratio Lectura/Escritura > 100:1.
Reviews	MongoDB	Documental / Lectura	Datos desestructurados. Alta escalabilidad horizontal. Integración sencilla con pipelines de agregación.
Analytics / Sessions	MongoDB	Write-Heavy (Series)	Soporte natural para documentos Time-Series y eliminación automática mediante índices TTL.

- **Análisis en profundidad del Teorema CAP y PACELC**

El Teorema CAP establece que, en un sistema distribuido propenso a particiones de red (P), es imposible garantizar simultáneamente la Consistencia (C) estricta y la Disponibilidad (A). Adicionalmente, el teorema PACELC expande esto indicando que, incluso cuando no hay particiones (E - Else), se debe elegir entre Latencia (L) y Consistencia (C).

La arquitectura híbrida aborda estos teoremas separando los dominios de datos para satisfacer diferentes necesidades del negocio:

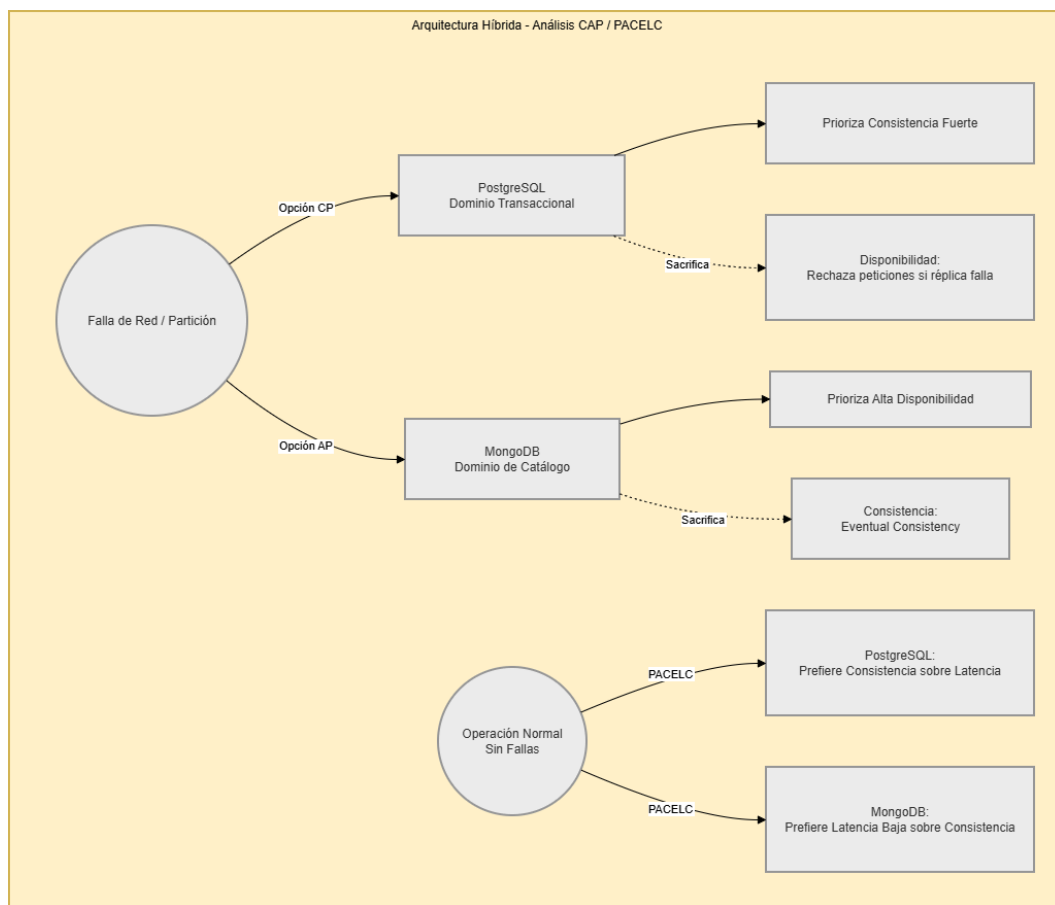


Figura 3. Análisis de Aplicación del Teorema CAP en el Diseño Híbrido Propuesto para el Dataset Ollist.

1. PostgreSQL: Enfoque CP y PC en(PACELC)

- Comportamiento ante particiones (P): Si el nodo primario pierde conexión con las réplicas síncronas necesarias para alcanzar el quórum, el clúster bloqueará nuevas escrituras (rechaza pagos/órdenes) para evitar estados divergentes. (Elige C, sacrifica A).
- Comportamiento normal (E): Se prefiere la Consistencia (C) sobre la Latencia (L). Cada escritura pasa por un proceso estricto de commit en el disco (WAL) antes de responder al cliente, garantizando ACID, aunque la latencia de escritura sea mayor.

2. MongoDB: Enfoque AP y PA/EL en (PACELC)

- Comportamiento ante particiones (P): Si un nodo secundario de MongoDB pierde conexión con el primario, continuará sirviendo lecturas de catálogo utilizando su data local, aunque esté potencialmente obsoleta. El usuario sigue navegando y comprando sin percibir caídas. (Elige A, sacrifica C).
- Comportamiento normal (E): Se prefiere la Baja Latencia (L) sobre la Consistencia (C). Se utilizan Secondary Reads para balancear la carga globalmente, permitiendo latencias de lectura <10ms, aceptando que un usuario podría ver un precio que cambió hace milisegundos.

- **Evaluación de Trade-offs y Estrategias de Mitigación**

La persistencia políglota introduce beneficios innegables para la escala, pero requiere gobernar la complejidad técnica. A continuación, detallamos los compromisos asumidos:

Decisión Arquitectónica (Trade-off)	Beneficio Esperado	Riesgo o Desventaja	Estrategia de Mitigación Detallada
<i>Separar PG y Mongo</i>	Soporta un ratio >100:1 de lecturas/escrituras. Latencia P95 < 50ms en catálogo.	Venta de productos agotados por desfase.	Soft Allocations & UX: Validar stock real solo al añadir al carrito o iniciar checkout. Implementar reservas temporales en Redis (ej. 5 mins) durante el pago. Mostrar mensajes UX como "Últimas unidades".
Desnormalización en MongoDB	Latencia ultrabaja. Se evitan JOINS y peticiones múltiples para renderizar una página de producto.	Si un vendedor cambia su nombre en PG, miles de productos en Mongo podrían mostrar el nombre antiguo.	PG es la única fuente de verdad. Se implementa un Flujo de Sincronización Estricto (CDC). Puesto que no se editan datos maestros directamente en MongoDB.
Consistencia Eventual Asíncrona	Bajo acoplamiento. PG no espera a Mongo para confirmar un guardado, mejorando la latencia de escritura.	Si el servicio de sincronización falla o la red cae, MongoDB queda desincronizado permanentemente.	Uso de una tabla <code>outbox_events</code> en la misma transacción de PG y una cola (Kafka/RabbitMQ) para garantizar At-Least-Once Delivery (entrega al menos una vez).
Operación de Dos Motores Diferentes	Uso de la herramienta adecuada para el trabajo adecuado (SQL para finanzas, NoSQL para catálogos).	Duplicación de esfuerzos en monitoreo, backups, despliegues y curva de aprendizaje del equipo.	Uso obligatorio de servicios gestionados (ej. Supabase para PG, Atlas para Mongo). Uso de infraestructura como Código (Terraform) y observabilidad centralizada.

- **Flujos de sincronización (PostgreSQL-MongoDB)**

Para mantener la cohesión de los datos sin crear un acoplamiento síncrono que degrade el rendimiento, es diseñada una arquitectura reactiva basada en la combinación del Patrón: Change Data Capture (CDC) con Patrón Outbox:

1. Origen y Transacción - PostgreSQL: Un vendedor actualiza las especificaciones de un producto. La transacción guarda los datos en la tabla `products` y, simultáneamente, escribe un evento en una tabla transaccional especial llamada `outbox_events` (garantizando que la actualización y la intención de notificar ocurran atómicamente).
2. Captura - Debezium: Debezium lee continuamente el Write-Ahead Log (WAL) de PostgreSQL, detectando los nuevos registros en la tabla `outbox_events`.
3. Distribución - Apache Kafka / EventBridge: El CDC emite el evento al tópic `catalog.product.updated`.

4. Consumo y Transformación - Worker Node: Un microservicio suscriptor recibe el evento, ejecuta la transformación necesaria (por ejemplo, adaptar el esquema rígido relacional al Attribute Pattern de Mongo) y ejecuta una operación de updateOne con la opción upsert: true en MongoDB.

Hipótesis: El catálogo en MongoDB se mantiene eventualmente consistente, en el orden de los milisegundos, aislando completamente el rendimiento de escritura transaccional de las pesadas consultas de lectura del portal de cara al cliente.

Anexos

- **Diccionario de datos**

Tabla	Campo	Tipo	Restricción	Descripción
customers	customer_id	UUID	PK	Identificador único
customers	email	VARCHAR(150)	UNIQUE	Correo del cliente
orders	order_id	UUID	PK	Pedido
orders	customer_id	UUID	FK	Cliente asociado
order_items	quantity	INTEGER	CHECK > 0	Cantidad
payments	payment_value	NUMERIC(12,2)	CHECK >=0	Valor pago

- **Scripts SQL preliminares**

- Creación de extensiones

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS hstore;
```

- Índices

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

```
CREATE INDEX idx_orders_date
ON orders(purchase_timestamp);
```

```
CREATE INDEX idx_products_specs
ON products
USING GIN(product_specifications);
```

```
CREATE INDEX idx_product_name_trgm
```

```
ON products
USING GIN(product_name gin_trgm_ops);
```

- Trigger `updated_at`

```
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

- Ejemplos de datos

- customers

```
INSERT INTO customers (
    customer_id,
    first_name,
    last_name,
    email
)
VALUES (
    gen_random_uuid(),
    'Pepito',
    'Perez',
    'pepito@email.com'
);
```

- orders

```
INSERT INTO orders (
    order_id,
    customer_id,
    order_status,
    purchase_timestamp,
    total_amount
)
VALUES (
    gen_random_uuid(),
    'CLIENT_UUID',
    'PAID',
    CURRENT_TIMESTAMP,
    1500000
);
```